

A Nearly Quartic Separation Between Randomized and Quantum Query Complexity

OpenAI

October 5, 2026

Abstract

We construct total Boolean functions with a nearly quartic separation between bounded-error randomized and quantum query complexity. Writing these complexities as $R(f)$ and $Q(f)$, the examples rule out every universal bound $R(f) = O((1 + Q(f))^\alpha)$ with $\alpha < 4$. Thus the known quartic upper bound has the optimal exponent, disproving the conjectured cubic bound. Both complexities count worst-case bit queries with error at most $1/3$ on every input.

1 Introduction

How much can a quantum algorithm reduce the number of input bits needed to compute a total Boolean function? Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$. We write $R(f)$ and $Q(f)$ for the minimum worst-case numbers of bit queries made by randomized and quantum algorithms that compute f with error at most $1/3$ on every input. Quantum queries use the standard oracle

$$O_x|i, b, z\rangle = |i, b \oplus x_i, z\rangle.$$

Computation between queries is unrestricted. In particular, all query bounds in this paper are worst-case bounds, including over internal randomness and measurement outcomes.

Polynomial relations between these two measures express a basic restriction on quantum speedups for total functions. To ask for the sharp power without imposing a particular logarithmic correction, define

$$\alpha_{\text{tot}} = \inf \left\{ \alpha \geq 0 : \begin{array}{l} \text{there exists } C_\alpha < \infty \text{ such that, for every } n \geq 1 \text{ and} \\ f : \{0, 1\}^n \rightarrow \{0, 1\}, \quad R(f) \leq C_\alpha (1 + Q(f))^\alpha \end{array} \right\}. \quad (1)$$

The additive 1 includes constant functions. We prove the following.

Theorem 1.1. *The optimal power is $\alpha_{\text{tot}} = 4$. More precisely, $R(f) = O((1 + Q(f))^4)$ for every total Boolean function. For each fixed integer $k \geq 2$, there are positive constants c_k, C_k, b_k and total Boolean functions $F_{k,m}$, for all sufficiently large powers of four m , such that*

$$Q(F_{k,m}) \leq C_k \sqrt{m} (\log m)^{b_k}, \quad R(F_{k,m}) \geq c_k \frac{m^{2-1/k}}{(\log m)^2}. \quad (2)$$

The universal upper bound is known. The contribution is the family in (2), which rules out every smaller power, including the conjectured cubic relation.

1.1 Historical context

Grover’s search algorithm [16] gives the basic quadratic speedup for the OR function, with a matching quantum lower bound [10]. Beals, Buhrman, Cleve, Mosca, and de Wolf [8] proved the general polynomial relation $D(f) = O(Q(f)^6)$, where $D(f)$ is exact deterministic query complexity. Their result builds on the combinatorial and polynomial measures studied by Nisan [20] and Nisan and Szegedy [21]. Thus totality rules out the exponential query speedups possible for partial functions, but it leaves a substantial range of possible powers.

Pointer constructions introduced by Göös, Pitassi, and Watson [15] and developed by Ambainis, Balodis, Belovs, Lee, Santha, and Smotrovs [4] gave a nearly quartic separation between deterministic and quantum queries. For bounded-error randomized queries, Aaronson, Ben-David, and Kothari [2] introduced the cheat-sheet framework and obtained a power-5/2 separation, up to logarithmic factors. This framework places certificates in an array whose correct address is determined by answers to partial-function instances. Improvements in those partial-function separations led to power $8/3 - o(1)$ by Tal [26], and then $3 - o(1)$ by Bansal and Sinha [6] and independently Sherstov, Storozhenko, and Wu [23].

On the upper-bound side, Aaronson, Ben-David, Kothari, Rao, and Tal [3], using Huang’s sensitivity theorem [17], proved $D(f) = O(Q(f)^4)$. They conjectured that randomized complexity satisfies the stronger relation $R(f) = O(Q(f)^3)$ [3, Conjecture 2]. Theorem 1.1 disproves that conjecture and shows that the quartic upper bound already has the optimal power for bounded-error randomized complexity.

There is also a direct connection to algorithms that compare candidate witnesses. Quantum minimum finding [14], sink finding [25], and king finding in tournaments [18] provide related search procedures. Recent constructions of Ben-David and Kothari [9] and Ambainis, Iraids, and Kokainis [5] use comparisons of candidate witnesses to separate certificate complexity from randomized and quantum query complexity. Certificate complexity measures the worst-case size of a smallest partial input assignment that fixes the function value. Their nearly quartic separations between certificate and quantum complexity concern a different classical measure, but their comparison-based viewpoint is particularly relevant here.

1.2 The construction and its two analyses

The input is organized into m columns. Each column i contains a string X_i of m bits and an array of certificate cells. The address of the relevant cell is the vector of answers to several disjoint instances of a partial function. Quantumly these answers are cheap to obtain on promised inputs; classically they are hard to guess. Each cell stores pointers to positions in all the other strings X_j , together with a circuit transcript certifying the address.

A column wins when its own string is all ones, its address is defined, and its certificate is correct and points to a zero in every other column. The total function asks whether a winner exists. There can be at most one. An incorrect address or transcript simply fails verification; the input to the total function carries no promise.

The direction of the pointers matters. A true winner can use its own certificate to disqualify every opponent. Conversely, no cell in any opponent can point to a zero in the winner’s all-one string. Thus a cheap quantum comparison favors the winner with high probability even if the opponent’s address instances violate their promises. Comparisons between two non-winners may have arbitrary probabilities. Section 3 proves a tournament lemma that permits precisely this behavior. It produces a short list containing the winner in $\sqrt{m}$ times a polylogarithmic number of queries, after which quantum search checks the certificates.

The classical proof starts with one independently hidden zero in every X_i and blank certificate cells. We choose a uniformly random column, erase its zero, and fill its correctly addressed cell. These two changes produce a winner. A randomized algorithm can distinguish the two inputs only by finding the erased zero or querying the filled cell. The first event is unlikely for a uniformly chosen column; the second requires either many address queries in that column or a correct guess of an entire vector of hard answers. An adaptive direct-product argument makes this last assertion precise without conditioning on the algorithm's choice of which columns to investigate.

The tournament analysis is useful independently of this construction. It uses the actual probabilities of comparisons as multiplicative weights; fresh coherent workspaces and amplitude amplification sample those weights exactly. Only comparisons involving a genuine winner need to be reliable. This extends the candidate-comparison approach to the arbitrary behavior caused here by off-promise address inputs. The classical analysis combines the planted input with a standard per-instance-capped direct-product argument [22, 13]; the separate cell arrays let this argument charge the cost of addressing many possible winners.

Section 2 defines the function and its certificates. Sections 3 and 4 prove the two bounds in (2); Section 5 derives the optimal power. Appendix A gives a self-contained proof of the known quartic upper bound. The only external lower-bound input is the Forrelation theorem stated in Section 2.

1.3 Conventions

We write $[m] = \{1, \dots, m\}$ and use logarithms to base two, except in expressions where changing the base only changes an absolute constant. Constants in bounds for $F_{k,m}$ may depend on the fixed integer k . A partial Boolean function is specified on a subset of $\{0, 1\}^N$; its algorithms must be correct on that subset. The quantum routines used here have a fixed query cap on every input, including inputs outside the promise. A circuit transcript lists proposed values of the non-input gates; the actual input bits are read directly when a gate check needs them.

2 An addressed certificate with outward pointers

We construct a total function whose accepting certificate belongs to one column. The certificate must certify the answers to several Forrelation instances and point to a zero in every other column. The latter requirement will let a genuine accepting column defeat any competitor, even when that competitor's Forrelation instances violate their promise.

Throughout the construction, fix an integer $k \geq 2$ and let m tend to infinity through powers of four. Constants in $O_k(\cdot)$ may depend on k , which remains fixed as m grows. Write $[m] = \{1, \dots, m\}$.

2.1 The partial function and its exact circuit

We use the k -fold Forrelation problem introduced by Aaronson and Ambainis [1], with the thresholds analyzed by Bansal and Sinha [6]. Let H be the normalized Walsh–Hadamard matrix of order m , and let $u = m^{-1/2}(1, \dots, 1)^\top$. We identify a bit a with the sign $(-1)^a$. For a sign vector z , write D_z for the diagonal matrix with diagonal z . Given k sign vectors $z_1, \dots, z_k \in \{\pm 1\}^m$, put

$$\Phi(z_1, \dots, z_k) = u^\top D_{z_1} H D_{z_2} H \cdots H D_{z_k} u, \quad \delta = 2^{-5k}. \quad (3)$$

Define the partial Boolean function $P = P_{k,m}$ on km bits by

$$P(z_1, \dots, z_k) = \begin{cases} 1, & \Phi(z_1, \dots, z_k) \geq \delta, \\ 0, & |\Phi(z_1, \dots, z_k)| \leq \delta/2, \end{cases} \quad (4)$$

and leave it undefined on all other inputs. The two displayed cases are disjoint. Query complexity for this partial function requires correctness only on its domain.

Lemma 2.1. *For each fixed integer $k \geq 2$, the functions $P_{k,m}$ have the following properties.*

(i) *For all sufficiently large powers of four m ,*

$$R(P_{k,m}) > L, \quad L = \left\lfloor \frac{m^{1-1/k}}{(\log_2 m)^2} \right\rfloor. \quad (5)$$

(ii) *For every $0 < \eta < 1/2$, a quantum algorithm uses $O_k(1 + \log(1/\eta))$ bit queries and returns $P_{k,m}$ with error at most η on every input in its domain. The algorithm can be implemented coherently, retaining its workspace. On every input, promised or not, it produces a bit when its output register is measured.*

(iii) *There is a Boolean circuit $C = C_{k,m}$ of fan-in at most two and size $S = S_{k,m} = O_k(m(\log m)^2)$ with two output bits d, b such that*

$$d = 1 \iff P_{k,m} \text{ is defined}, \quad d = 1 \implies b = P_{k,m}.$$

The circuit computes these outputs exactly on every input.

Proof. Theorem 1.3 and Corollary 1.4 of Bansal and Sinha [6] apply to exactly the normalized form and thresholds in (3)–(4). They give, for fixed k and error $1/3$,

$$R(P_{k,m}) = \Omega_k \left(\left(\frac{m}{\log(km)} \right)^{1-1/k} \right).$$

This exceeds the value of L in (5) for all sufficiently large m .

For the quantum algorithm, prepare u , apply the unitary product in (3), in its right-to-left order, and test the projection onto u . Each D_{z_h} uses one phase query, implemented by the standard bit oracle, while H and the projection require no queries. The projection succeeds with probability Φ^2 . This probability is at least δ^2 on a 1-input and at most $\delta^2/4$ on a 0-input. Independent repetitions and a threshold between these two numbers give error η with $O_k(1 + \log(1/\eta))$ repetitions. The measurement outcomes and the final threshold decision can instead be computed into registers and retained, giving the claimed coherent implementation.

For the circuit, let $W = \sqrt{m}H$, whose entries are ± 1 . The scalar

$$N = z_1^\top W D_{z_2} W \cdots W D_{z_k} (1, \dots, 1)^\top$$

is an integer, and

$$\Phi = \frac{N}{M_{k,m}}, \quad M_{k,m} = m(\sqrt{m})^{k-1}.$$

The denominator is an integer because m is a power of four. Compute N by $k-1$ unnormalized Walsh–Hadamard transforms, coordinatewise sign changes, and a final signed sum. Each transform uses $O(m \log m)$ additions and subtractions by the usual butterfly recursion. All intermediate integers have absolute value at most $m^{O(k)}$, so $O_k(\log m)$ bits suffice. Binary additions, subtractions, sign changes, and comparisons have fan-in-two Boolean circuits of size linear in this bit length. Consequently the entire arithmetic circuit has Boolean size $O_k(m(\log m)^2)$.

Finally, the two tests in (4) are the exact integer comparisons

$$2^{5k} N \geq M_{k,m}, \quad 2^{5k+1} |N| \leq M_{k,m}.$$

Set d to the disjunction of these comparisons and b to the first comparison. This adds only $O_k(\log m)$ gates and proves the last assertion. $\square$

The exact circuit serves a different purpose from the quantum algorithm. The algorithm will propose an address. A transcript of the circuit will certify that this address consists of defined, correct answers, including on inputs where an arbitrary proposed address has no reason to be correct.

2.2 The total function

Set

$$r = \lceil 20 \log_2 m \rceil, \quad \ell = \log_2 m. \quad (6)$$

The input has m columns. Column $i \in [m]$ contains the following bits:

- (i) A string $X_i \in \{0, 1\}^m$.
- (ii) Strings $A_{i,1}, \dots, A_{i,r} \in \{0, 1\}^{km}$, with no promise imposed on them.
- (iii) A cell $B_{i,s}$ for each address $s \in \{0, 1\}^r$. This cell contains a pointer $p_{i,s,j} \in [m]$ for each $j \neq i$, encoded by ℓ bits using a fixed bijection, and an alleged evaluation transcript of $C_{k,m}$ on each $A_{i,h}$, $h \in [r]$.

An evaluation transcript consists of one bit for every non-input gate of $C_{k,m}$, including its output gates. Circuit inputs are the actual bits of $A_{i,h}$; they need not be copied into the transcript. We count these non-input gates in S .

Call column i a *winner* if $X_i = 1^m$, every $P(A_{i,h})$ is defined, and the cell at its true address

$$s_i = (P(A_{i,1}), \dots, P(A_{i,r})) \quad (7)$$

contains correct evaluation transcripts and pointers satisfying

$$X_j[p_{i,s_i,j}] = 0 \quad \text{for every } j \neq i. \quad (8)$$

Define $F_{k,m}$ to be 1 if a winner exists, and 0 otherwise. This is a total Boolean function: all the preceding conditions are predicates on the input bits, and no promise is made about the A -strings, the pointers, or the transcripts.

The addressed transcripts have the certificate role used in cheat-sheet constructions [2], while pointers between columns also occur in earlier query-complexity separations [15, 4]. Here the pointers have a specific outward orientation: an accepting column supplies a direct zero witness against every competitor. In particular, there can be at most one winner, since a winner's pointer to another column certifies that the latter is not all ones. Figure 1 depicts this condition.

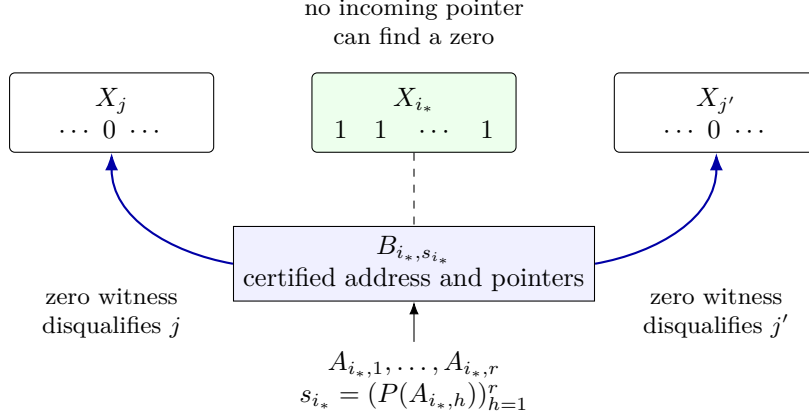


Figure 1: The relevant cell of a winning column points outward to a zero in every other column; two such pointers are shown. Only the indicated zeros are prescribed in the other columns. The cell's address is computed from the partial-function answers and certified by stored circuit transcripts. The winner's all-one string prevents every opponent from disqualifying it, even at an incorrectly decoded address.

We next make certificate verification precise. For a specified pair (i, s) , say that i wins *at address* s if i is a winner and $s = s_i$.

Lemma 2.2. *For every input to $F_{k,m}$ and every pair $(i, s) \in [m] \times \{0, 1\}^r$, the assertion that i wins at address s is a conjunction of*

$$K_{k,m} = m + (m - 1) + r(S_{k,m} + 2) = O_k(m(\log m)^3)$$

Boolean tests. Each test can be evaluated exactly with $O(\log m)$ bit queries. The tests can also be evaluated coherently with the same query bound, with their indices supplied in superposition. Their number and indexing convention are the same for every (i, s) .

Proof. Use one test for each bit of X_i , requiring it to be 1, and one test for each $j \neq i$, reading the pointer $p_{i,s,j}$ and requiring the indicated bit of X_j to be 0. The latter uses $\ell + 1$ queries.

For each of the r transcripts in $B_{i,s}$, use one test per gate, requiring its claimed value to be the prescribed Boolean function of its predecessors. A predecessor value is read from the transcript for a non-input gate and from $A_{i,h}$ for an input wire. Fan-in at most two therefore makes each such test use at most three queries. Add two output tests, requiring $d = 1$ and $b = s_h$. By induction in a topological order of the circuit, all gate tests pass if and only if the transcript is the true evaluation. The output tests then say exactly that $P(A_{i,h})$ is defined and equals s_h . Together with the X_i and pointer tests, these are precisely the winner conditions at (i, s) .

The wiring of the circuit and the locations of all bits are fixed and require no queries to compute. A test can load the required bits into ancillas, compute its result reversibly, and unload the bits by querying again. The bit oracle is its own inverse. A controlled use of that oracle, when needed, is likewise implemented by loading, controlled use of the loaded value, and unloading. Thus coherent evaluation, including an input-dependent pointer lookup, changes the query count by at most a constant factor. $\square$

Two consequences will be used repeatedly. A winning column has one address at which every test passes; a nonwinning column has a failing test at every address. Thus an address proposed on an unpromised input can always be checked against the total definition.

3 Quantum search with probabilistic comparisons

We first prove a general tournament statement. A comparison need not behave almost deterministically: its outcome probabilities may be arbitrary, provided one distinguished candidate beats each opponent with sufficiently high probability. This is the additional feature needed beyond comparison algorithms for a fixed tournament [25, 18, 9]. There is no promised gap between a nonwinner comparison probability and $1/2$, so ordinary error reduction does not provide uniformly cheap bounded-error access to a fixed tournament. After proving the statement, we construct the comparisons for $F_{k,m}$ and verify the candidates they produce.

3.1 Amplification without changing the sampled distribution

The tournament will sample a candidate conditioned on defeating every previously selected candidate. Amplification must preserve that conditional distribution while increasing the probability of obtaining a sample. This is a direct consequence of the two-dimensional rotation underlying amplitude amplification [11, 12]; we give the details because the distribution, as well as the success probability, is used in the tournament.

Lemma 3.1 (Bounded amplification). *Let A be a quantum state preparation with a designated success bit. Its success probability is p , and, if $p > 0$, its normalized successful state is $|g\rangle$. Let $0 < \gamma \leq 1/2$ and $0 < \eta < 1/2$. There is a procedure using*

$$O(\gamma^{-1/2} \log(1/\eta))$$

calls to A and A^{-1} that either reports failure or produces a successful outcome. Conditional on success, the output state is $|g\rangle$ up to a global phase. If $p \geq \gamma$, its failure probability is at most η . If $p = 0$, it always reports failure. The call bound holds for every p and on every branch.

Proof. For $0 < p < 1$, write

$$A|0\rangle = \sin \theta |g\rangle + \cos \theta |b\rangle, \quad 0 < \theta < \pi/2,$$

where the two normalized vectors lie in the successful and unsuccessful subspaces. Reflection about $A|0\rangle$, followed in the appropriate order by a phase flip of the success bit, gives after ℓ Grover iterations the state

$$\sin((2\ell + 1)\theta) |g\rangle + \cos((2\ell + 1)\theta) |b\rangle.$$

Thus measuring success leaves exactly the same normalized successful state, independently of ℓ . Each reflection about the prepared state uses A , a query-free reflection about $|0\rangle$, and A^{-1} .

Set $M = \lceil 10/\sqrt{\gamma} \rceil$. A trial chooses, with equal probabilities, either $\ell = 0$ or a uniform $\ell \in \{0, \dots, M-1\}$, then measures the success bit. If $p \geq 1/2$, the first choice alone gives success probability at least $1/4$. If $\gamma \leq p \leq 1/2$, then

$$\sin(2\theta) = 2\sqrt{p(1-p)} \geq \sqrt{\gamma}.$$

Summing a geometric progression bounds the averaged success probability in the second choice by

$$\frac{1}{M} \sum_{\ell=0}^{M-1} \sin^2((2\ell + 1)\theta) \geq \frac{1}{2} - \frac{1}{2M|\sin(2\theta)|} \geq \frac{9}{20}. \quad (9)$$

Every trial therefore succeeds with probability at least $9/40$ when $p \geq \gamma$. Repeat a fixed $\lceil \log(1/\eta)/\log(40/31) \rceil$ number of trials with fresh registers, stopping at the first success. The

probability that all fail is at most η . Each trial uses $O(M)$ calls, whether it succeeds or fails. Every successful trial has the same state $|g\rangle$, so neither the random choice of length nor stopping at the first success changes that state. The cases $p = 0$ and $p = 1$ follow directly, with the successful component absent or equal to the entire prepared state. $\square$

3.2 The tournament

Fix an input to the bit oracle. For candidates in $[m]$, suppose a coherent comparison produces one victorious candidate for each unordered pair. It retains all workspace, so it is a unitary state preparation that can be reversed. Let a_{ij} be the probability that i defeats j when the comparison result is measured. We always use the same circuit for the unordered pair, and set $a_{ii} = 0$. Hence

$$a_{ij} + a_{ji} = 1 \quad (i \neq j), \quad a_{ii} = 0. \quad (10)$$

The following result is pointwise in the fixed oracle input; its probabilities are over the algorithm's internal randomness and measurements.

Proposition 3.2 (A tournament for probabilistic comparisons). *For $m \geq 2$, put*

$$T = \lceil \log(1000m) \rceil, \quad \epsilon = \frac{1}{1000T}.$$

Suppose a coherent comparison circuit, controlled by two candidate indices and reversible with all workspace retained, uses at most $c \geq 1$ bit queries. Let a_{ij} be the probability that it reports i defeating j , and assume (10). There is a quantum algorithm making

$$O(c\sqrt{m}T^2 \log T)$$

queries on every branch and returning a list of at most T candidates with the following guarantee. If $i_ \in [m]$ satisfies*

$$a_{i_*j} \geq 1 - \epsilon \quad (j \neq i_*), \quad (11)$$

then the list contains i_ with probability at least $1 - 2m/2^T - T\epsilon$.*

Proof. We analyze an ideal sampling process and then implement each of its steps quantumly. Start with $w_i = 1$ for all i , and write $W = \sum_i w_i$. When $W > 0$, sample a pivot p with probability w_p/W , append it to the list, and update

$$w_i \leftarrow w_i a_{ip} \quad (i \in [m]). \quad (12)$$

When $W = 0$, leave the list unchanged and keep all weights zero. The weights only define this ideal process; the algorithm will not compute them numerically.

For $W > 0$, complementarity gives the exact drift identity

$$\begin{aligned} \mathbb{E}[W_{\text{new}} \mid w] &= \frac{1}{W} \sum_{p,i} w_p w_i a_{ip} \\ &= \frac{W^2 - \sum_i w_i^2}{2W} \leq \frac{W}{2}. \end{aligned} \quad (13)$$

The same inequality holds when $W = 0$, so after T steps $\mathbb{E}W_T \leq m/2^T$. If a candidate i_* satisfying (11) has never been selected, its final weight is at least

$$(1 - \epsilon)^T \geq 1 - T\epsilon > \frac{1}{2}.$$

In particular, the process cannot reach zero total weight before selecting it. Markov's inequality implies

$$\Pr[i_* \text{ absent from the ideal list}] \leq \Pr[W_T \geq 1/2] \leq \frac{2m}{2^T}. \quad (14)$$

Now fix an arbitrary list of previously selected pivots $p_1, \dots, p_t$, where $t < T$. Prepare a uniform superposition of $i \in [m]$, and, for each pivot, compare i with that pivot in a fresh workspace. Conditional on i , let $|\phi(i, p_b)\rangle$ denote the complete comparison state, including its result bit. Before computing a success bit, the joint state is

$$\frac{1}{\sqrt{m}} \sum_{i=1}^m |i\rangle \bigotimes_{b=1}^t |\phi(i, p_b)\rangle. \quad (15)$$

Set the success bit precisely when every comparison says that i defeats its pivot. All garbage is retained. By the tensor product in (15), the conditional success probability for index i is

$$\prod_{b=1}^t a_{ip_b} = w_i.$$

The empty product is one. Distinct index registers are orthogonal, so the total success probability is W/m , and the conditional distribution of i on success is exactly w_i/W .

Apply Lemma 3.1 with $\gamma = 1/(2m)$ and $\eta = \epsilon$. A successful sample still has exactly this index distribution: amplification preserves the full successful state, not just its total probability. If sampling reports failure, stop and return the list already accumulated. Otherwise measure the index, append it, and continue, for at most T stages.

Until i_* is first selected, its weight is at least $1/2$, so $W/m \geq 1/(2m)$. At each such history the sampler has failure probability at most ϵ and, conditional on success, exactly the ideal pivot law. One can therefore couple that stage to an ideal pivot, with probability at most ϵ of terminating instead. The coupling is conditional on the entire previous pivot history; it does not require history-independent failure probabilities. Summing these failure probabilities over at most T stages and using (14) gives the stated guarantee. Once i_* has entered the list, later termination cannot remove it.

At stage t , the preparation and its inverse use $O(c(t+1))$ queries. Lemma 3.1 uses $O(\sqrt{m} \log(1/\epsilon))$ such calls. Summing over $t < T$ gives $O(c\sqrt{m}T^2 \log T)$ queries. The number of stages and all trial counts are fixed upper bounds, so this is a worst-case query bound even if there is no candidate satisfying (11). $\square$

The proof uses no order or transitivity among the candidates. Its shrinking quantity is the sum of fractional weights in (12), and complementarity alone yields (13). This is why unpromised address computations will be permitted in the application.

3.3 Comparing columns and verifying the list

We now fix an arbitrary input to $F_{k,m}$, including possibly undefined Forrelation instances and incorrect circuit transcripts. Keep T and ϵ as in Proposition 3.2. To decode column i , run the quantum algorithm for each of its r partial-function instances, amplified to error at most ϵ/r . Repetition and a majority computation take

$$O_k(r \log(r/\epsilon)) = (\log m)^{O_k(1)}$$

queries. If all these instances are defined, the resulting address is correct with probability at least $1 - \epsilon$. Otherwise the routine still returns some address. All repetitions and majority computations can be performed unitarily with their workspaces retained.

For a pair of distinct columns, sort their indices and apply the following comparison in this fixed order. Decode the address of each column in a separate workspace. Read each column's pointer toward the other at its decoded address, and inspect the bit of the other's X string to which it points. A column *disqualifies* its opponent if that bit is zero. If exactly one column disqualifies the other, declare it victorious; otherwise declare the smaller index victorious. For a self-comparison, return false without reading a pointer.

These operations prepare a comparison state with $c = (\log m)^{O_k(1)}$ queries. The decoded addresses remain in their registers, and subsequent operations are controlled in the computational basis. Consequently its measurement probabilities agree with the description just given; different address outcomes are orthogonal and do not interfere. Reversing the complete preparation costs the same number of bit queries. Using the same sorted-pair circuit for either ordering ensures (10).

If i_* is a winner, then X_{i_*} is all ones. No opponent, at any decoded address, can point to a zero there. When the address of i_* is correctly decoded, its own certificate points to a zero in every other column. Thus

$$a_{i_*j} \geq 1 - \epsilon \quad (j \neq i_*),$$

regardless of whether the opponent's address instances are defined. Proposition 3.2 therefore returns a list containing the winner, when one exists, with probability at least $1 - 2m/2^T - T\epsilon$.

For each listed column, decode and measure its address afresh. At that fixed address, the winner conditions are the exact local tests of Lemma 2.2. Let their number be $K = m(\log m)^{O_k(1)}$. A uniform test index and an exact failing-test flag give a preparation whose success probability is zero when all tests pass, and at least $1/K$ otherwise. Lemma 3.1, with $\gamma = 1/K$ and $\eta = \epsilon$, searches for a failed test in

$$O(\sqrt{K} \log(1/\epsilon) \log m) = \sqrt{m}(\log m)^{O_k(1)}$$

queries. Here $K \geq m \geq 4$. Reject this candidate if a failed test is found, and accept it otherwise. Exact testing cannot report a failure when all tests pass. If the column is not a winner, every address has a failed test, so it is accepted with probability at most ϵ . A true winner is accepted whenever decoding is correct: in that case there is no failed test for the search to report. Its rejection probability is therefore at most ϵ , solely from address decoding.

Output one if any listed candidate is accepted. On a zero-input, a union bound gives error at most $T\epsilon$. On a one-input, the miss probability and the winner's verification error give total error at most

$$\frac{2m}{2^T} + T\epsilon + \epsilon < \frac{1}{3}.$$

The comparisons, tournament, and at most T verifications prove

Proposition 3.3. *For every fixed $k \geq 2$, the total functions of Section 2 satisfy*

$$Q(F_{k,m}) \leq \sqrt{m}(\log m)^{O_k(1)}.$$

All superposed array accesses used above are ordinary bit queries. The global bit index is computed reversibly from the column, cell, and wire indices. A pointer is read one bit at a time in $O(\log m)$ queries, after which its target index is computed without queries. A controlled bit query can be implemented by loading the addressed input bit into an ancilla, using it under the required control, and unloading it with a second query. Thus neither coherent addressing nor retaining and reversing comparison garbage changes the query model.

4 The randomized lower bound

We compare a distribution with one zero in every column to a coupled distribution in which one uniformly chosen column is made a winner. An algorithm can distinguish them only by finding the zero that was removed or by reaching the newly filled certificate cell. The first event requires searching many columns; the second requires either many Forrelation queries in the chosen column or a successful guess of its entire address.

4.1 Independent answers under adaptive queries

We first record the precise amplification fact needed for address guessing. We use the capped direct-product argument developed by Drucker [13, Section 1.3 and Lemmas 3.1–3.2], following earlier work of Shaltiel [22]. Its query limit is imposed separately on each input, while queries to different inputs may be interleaved adaptively.

Lemma 4.1. *Let $P: \mathcal{D} \rightarrow \{0, 1\}$, where $\mathcal{D} \subseteq \{0, 1\}^n$, let μ be a distribution on $\mathcal{D}$, and let L be a nonnegative integer. Suppose every deterministic algorithm making at most L queries computes P under μ with success probability at most β . If $U_1, \dots, U_r$ are independent samples from μ , every classical algorithm making at most L queries to each U_h has probability at most β^r of outputting*

$$(P(U_1), \dots, P(U_r)).$$

The algorithm may be randomized, choose its queries adaptively among the r inputs, and receive arbitrary additional information independent of $(U_1, \dots, U_r)$.

Proof. Fix the additional information and all internal random coins, leaving a deterministic algorithm. At any transcript τ of positive probability, the conditional law of $(U_1, \dots, U_r)$ is a product of their separate conditional laws. Indeed, the event of obtaining τ requires exactly the observed bit values in each input. Any tuple satisfying these requirements follows the same query schedule, since the next query is determined by the preceding answers. Thus this event is a Cartesian product of constraints on the individual inputs.

For each h , let q_h be the number of queries already made to U_h , and let $V_h(\tau)$ be the largest success probability for computing $P(U_h)$ from its current conditional law using at most $L - q_h$ further queries. This maximum is attained because the domain and the remaining query budget are finite. If the next query addresses U_h , optimality gives

$$\mathbb{E}[V_h(\tau') \mid \tau] \leq V_h(\tau), \tag{16}$$

where τ' includes its answer: making that particular query and then using an optimal continuation is one allowed strategy in the definition of $V_h(\tau)$. The conditional laws and budgets of the other inputs do not change. Hence

$$\mathbb{E} \left[\prod_{j=1}^r V_j(\tau') \mid \tau \right] \leq \prod_{j=1}^r V_j(\tau).$$

The product is therefore a supermartingale over the successive queries. There are at most rL such queries; a run that stops earlier can be extended by steps leaving all values unchanged.

At the final transcript, the conditional probability that the algorithm's chosen answer vector is correct is at most

$$\prod_{h=1}^r \max_{b \in \{0,1\}} \Pr(P(U_h) = b \mid \tau) \leq \prod_{h=1}^r V_h(\tau).$$

The first inequality uses the product conditional law, and the second uses the option of making no further queries. Initially every V_h is at most β . Taking expectations proves the bound β^r . Averaging over the fixed information and coins proves the full statement. $\square$

Return to $P = P_{k,m}$ and L from (5), with m large enough that $1 \leq L < R(P)$. There are finitely many deterministic decision trees of depth at most L on this finite domain. A randomized algorithm of that query depth is a distribution on these trees. Finite minimax, in its query-complexity formulation [27], applied to the payoff that the answer is correct, gives

$$\begin{aligned} & \max_{\text{distributions on depth-}L \text{ trees}} \min_{x \in \text{dom } P} \Pr(\text{correct on } x) \\ &= \min_{\mu} \max_{\text{depth-}L \text{ trees } T} \Pr_{x \sim \mu}(T(x) = P(x)). \end{aligned}$$

The left-hand side is less than $2/3$, since its maximum is attained and $L < R(P)$. Consequently there is a distribution μ on $\text{dom } P$ for which every such deterministic tree has success probability at most $2/3$. Lemma 4.1 therefore bounds the success probability of guessing r independent answers by $(2/3)^r$.

4.2 Coupled zero and one inputs

Proposition 4.2. *For each fixed integer $k \geq 2$ and every sufficiently large power of four m , the total function of Section 2 satisfies*

$$R(F_{k,m}) > G := \left\lfloor \frac{mL}{100} \right\rfloor, \quad L = \left\lfloor \frac{m^{1-1/k}}{(\log_2 m)^2} \right\rfloor. \quad (17)$$

Proof. Generate a *dummy input* $Y^{(0)}$ as follows. Independently for each $i \in [m]$, choose J_i uniformly from $[m]$ and put a single zero at position J_i in X_i , setting its other bits to 1. Draw all $A_{i,h}$ independently from μ , independently also of the J_i . Set all cells $B_{i,s}$ to one fixed blank bit pattern. Since each X_i has a zero, $F_{k,m}(Y^{(0)}) = 0$.

Choose I uniformly from $[m]$, independently of the entire dummy input. Form a *planted input* $Y^{(1)}$ by setting $X_I = 1^m$ and filling its true-address cell B_{I,s_I} with the correct circuit transcripts and the pointers $p_{I,s_I,j} = J_j$ for $j \neq I$. All A -strings are in the domain of P , so s_I is well defined. Column I is a winner, and therefore $F_{k,m}(Y^{(1)}) = 1$.

Fix a deterministic algorithm making at most G queries, and consider its run on $Y^{(0)}$. The coupled inputs can differ only at the bit $X_I[J_I]$ and inside the cell B_{I,s_I} . Until one of these locations is queried, the two executions have identical answers and hence identical subsequent queries. It follows that the difference between their output-1 probabilities is at most

$$\Pr(\text{the dummy run queries } X_I[J_I]) + \Pr(\text{the dummy run queries any bit of } B_{I,s_I}). \quad (18)$$

Counting every visit to the indicated cell is an upper bound even if the queried bit happens to agree with its blank value.

Finding the removed zero. Let Z be the number of columns whose zero is discovered in the dummy run. Before the zero in a column is found, its location is uniform among the positions in that column not yet queried. This remains true conditional on the full adaptive transcript: the other X -strings and all A -strings are independent of its zero location, and the blank cells supply no information about that location.

Call a discovery early if fewer than $m/2$ distinct positions in its column were queried before it. Conditional on the preceding transcript, each query has probability at most $2/m$ of an early discovery. Thus the expected number of early discoveries is at most $2G/m$. Each remaining discovery occurs in a column that receives at least $m/2$ queries, and there are at most $2G/m$ such columns. Hence

$$\mathbb{E}Z \leq \frac{4G}{m}.$$

The dummy run is independent of I , so

$$\Pr(\text{the dummy run queries } X_I[J_I]) = \frac{\mathbb{E}Z}{m} \leq \frac{4G}{m^2}. \quad (19)$$

Reaching the planted cell. Call a column heavy if, over the entire dummy run, the algorithm makes more than L queries in total to its strings $A_{i,1}, \dots, A_{i,r}$. There are at most G/L heavy columns. Independence of I gives

$$\Pr(I \text{ is heavy}) \leq \frac{G}{mL}. \quad (20)$$

For the other columns we use Lemma 4.1. The following simulation bounds a joint event and does not condition the input distribution on a column being nonheavy.

Fix $i \in [m]$ and a time $t \in \{1, \dots, G\}$. Give a simulator all the dummy input outside $A_{i,1}, \dots, A_{i,r}$ for free. This information is independent of those r strings. The simulator runs the deterministic algorithm up to its proposed t th query, but aborts before making an $(L+1)$ st query to these r strings in total. If it reaches time t and the proposed query is to a cell $B_{i,s}$, it outputs the address s as its guess for the vector of r answers. Otherwise, including if the simulated run stops or aborts, it outputs any fixed vector.

The simulator makes at most L queries in total to the target strings, and in particular at most L to each one. By Lemma 4.1, its probability of guessing s_i correctly is at most $(2/3)^r$. Whenever the original dummy run queries B_{i,s_i} at time t and column i is nonheavy, this simulator has not aborted and outputs the correct address. Thus

$$\Pr(i \text{ is nonheavy and the } t\text{th query is in } B_{i,s_i}) \leq (2/3)^r. \quad (21)$$

Union bounding over t and averaging over the independent uniform choice of I , then adding (20), yields

$$\Pr(\text{the dummy run queries } B_{I,s_I}) \leq \frac{G}{mL} + G(2/3)^r. \quad (22)$$

Equations (18), (19), and (22) bound the difference of output probabilities by

$$\frac{4G}{m^2} + \frac{G}{mL} + G(2/3)^r. \quad (23)$$

For all sufficiently large m , we have $1 \leq L \leq m$ and $G \leq mL/100$. The first two terms are therefore at most $4/100$ and $1/100$, respectively. Moreover, (6) gives

$$G(2/3)^r \leq \frac{1}{100} m^{2-20\log_2(3/2)} = o(1).$$

In particular, (23) is strictly less than $1/3$ for large m .

The same bound holds for a randomized algorithm of worst-case query cost G , by fixing its coins and then averaging. Such an algorithm, if it had worst-case error at most $1/3$, would output 1 with probability at most $1/3$ on $Y^{(0)}$ and at least $2/3$ on $Y^{(1)}$. Their output-1 probabilities would differ by at least $1/3$, contradicting the preceding estimate. Hence $R(F_{k,m}) > G$. $\square$

For fixed k , Proposition 4.2 implies

$$R(F_{k,m}) = \Omega_k \left(\frac{m^{2-1/k}}{(\log m)^2} \right).$$

The heavy-column estimate pays for learning an address, while the independent-answer estimate makes guessing an unlearned address negligible even over all G adaptive queries.

5 The optimal power

We combine the quantitative bounds before recalling the universal upper bound in the appendix.

Proof of Theorem 1.1. For each fixed $k \geq 2$, Proposition 3.3 gives $Q(F_{k,m}) \leq C_k \sqrt{m} (\log m)^{b_k}$ for suitable constants. Proposition 4.2 gives

$$R(F_{k,m}) > \left\lfloor \frac{m}{100} \left\lfloor \frac{m^{1-1/k}}{(\log m)^2} \right\rfloor \right\rfloor \geq c_k \frac{m^{2-1/k}}{(\log m)^2}$$

for sufficiently large m , after decreasing $c_k > 0$. This proves (2).

Now fix $0 \leq \alpha < 4$ and choose an integer $k \geq 2$ such that $2 - 1/k > \alpha/2$. The integer k remains fixed as m tends to infinity. The preceding bounds imply

$$\frac{R(F_{k,m})}{(1 + Q(F_{k,m}))^\alpha} \geq c_{k,\alpha} \frac{m^{2-1/k-\alpha/2}}{(\log m)^{2+\alpha b_k}} \rightarrow \infty.$$

Hence no exponent smaller than four belongs to the set in (1). Conversely, Theorem A.1 proves $R(f) \leq D(f) = O((1 + Q(f))^4)$ for every total Boolean function. Therefore four does belong to that set, and its infimum is four. $\square$

Separate address arrays force a classical algorithm to invest in many columns, while quantum comparisons locate a candidate before its certificate is checked. Quantum search then supplies the quadratic saving in checking the local tests. The direct-product estimate prevents a classical algorithm from bypassing the addressing cost by guessing certificate locations.

A The universal quartic bound

For completeness, we prove the upper bound used in Theorem 1.1. The argument is the one of Aaronson, Ben-David, Kothari, Rao, and Tal [3], combining Huang's signed-cube method [17] with the adversary bound and the deterministic degree–block-sensitivity inequality. No part of the construction is used in this appendix.

For a total Boolean function f , let $D(f)$ be its exact deterministic query complexity and $\deg(f)$ its degree as a real multilinear polynomial. The degree is unchanged by replacing input or output bits by signs. A nonempty set $B \subseteq [n]$ is a *sensitive block at x* if flipping every bit in B changes $f(x)$. The block sensitivity $bs(f)$ is the maximum, over x , of the number of pairwise disjoint sensitive blocks at x .

Theorem A.1 ([3]). *Every total Boolean function satisfies*

$$R(f) \leq D(f) = O((1 + Q(f))^4).$$

We establish the three ingredients separately. All matrix norms below are operator norms for the Euclidean norm.

Lemma A.2 (Nonnegative spectral adversary bound). *Let f be a nonconstant total Boolean function. Let Γ be a nonzero real symmetric matrix, indexed by inputs, with nonnegative entries and $\Gamma_{xy} = 0$ whenever $f(x) = f(y)$. Let*

$$(\Gamma_i)_{xy} = \Gamma_{xy} \mathbf{1}_{\{x_i \neq y_i\}}.$$

Then, for an absolute constant $c > 0$,

$$Q(f) \geq c \frac{\|\Gamma\|}{\max_i \|\Gamma_i\|}.$$

Proof. This is the spectral adversary argument of Barnum, Saks, and Szegedy [7]; see also Špalek and Szegedy [24]. Purify all randomness and defer measurements. A bounded-query algorithm can be represented with a fixed number of queries, with at most a constant-factor overhead: branches that have finished can perform ineffective queries, and controlled queries can be simulated with two bit queries. Let $|\psi_x^t\rangle$ be its normalized pure state on input x after t queries.

By the Perron–Frobenius theorem, Γ has a nonnegative unit eigenvector v of eigenvalue $\|\Gamma\|$. Track

$$W_t = \sum_{x,y} v_x v_y \Gamma_{xy} \langle \psi_x^t | \psi_y^t \rangle.$$

Initially the states are input-independent, so $W_0 = v^\top \Gamma v = \|\Gamma\|$. Input-independent unitaries preserve every overlap.

Immediately before a query, write the normalized state as

$$|\psi_x\rangle = \sum_i |i\rangle |\phi_{x,i}\rangle, \quad \sum_i \|\phi_{x,i}\|^2 = 1.$$

The bit oracle acts identically on inputs x, y at that index unless $x_i \neq y_i$. When they differ, the overlap changes in magnitude by at most $2\|\phi_{x,i}\|\|\phi_{y,i}\|$, since the difference between the relevant unitary and the identity has norm at most two. Put $u_i(x) = v_x \|\phi_{x,i}\|$. Nonnegativity of the weights gives

$$\begin{aligned} |W_{t+1} - W_t| &\leq 2 \sum_i u_i^\top \Gamma_i u_i \\ &\leq 2 \max_i \|\Gamma_i\| \sum_i \|u_i\|^2 = 2 \max_i \|\Gamma_i\|. \end{aligned}$$

The last equality follows from normalization of each query state and $\sum_x v_x^2 = 1$.

At the end, measure the output bit. For opposite-output inputs, one output-one probability is at most $1/3$, and the other is at least $2/3$. Splitting the two states by that output and applying Cauchy–Schwarz bounds their overlap in absolute value by $2\sqrt{2}/3$. Hence the final value satisfies

$$|W_{\text{final}}| \leq \frac{2\sqrt{2}}{3} \sum_{x,y} v_x v_y \Gamma_{xy} = \frac{2\sqrt{2}}{3} \|\Gamma\|.$$

The required total change in W divided by the per-query bound proves the assertion. $\square$

Let S_f be the adjacency matrix of the *sensitivity graph*: its vertices are $\{0, 1\}^n$, and two vertices are adjacent if they differ in exactly one bit and have different function values. Write $\lambda(f) = \|S_f\|$.

Lemma A.3. *For every nonconstant total Boolean function,*

$$\text{bs}(f) = O(Q(f)^2), \quad \lambda(f) = O(Q(f)).$$

Proof. Choose an input x with $b = \text{bs}(f)$ disjoint sensitive blocks. Take Γ to be the adjacency matrix of the star whose center is x and whose leaves are the b inputs obtained by flipping those blocks, with zero entries elsewhere. Its norm is $\sqrt{b}$. Each coordinate belongs to at most one block, so Γ_i has at most one undirected edge and norm at most one. Lemma A.2 gives $Q(f) = \Omega(\sqrt{b})$. This recovers the usual block-sensitivity lower bound for quantum queries [8].

For the second assertion, use $\Gamma = S_f$. The edges in any one coordinate form a partial matching, whose adjacency matrix has norm at most one. Another application of Lemma A.2 gives $Q(f) = \Omega(\lambda(f))$, as in [3]. $\square$

Lemma A.4 ([3], using [17]). *Every nonconstant total Boolean function satisfies $\deg(f) \leq \lambda(f)^2$.*

Proof. Write the function with sign inputs and sign outputs, and set $d = \deg(f) \geq 1$. Choose a monomial supported on a set of d variables with nonzero coefficient, and fix all other variables to any signs. Its coefficient survives: any different monomial that could contribute to that coefficient would strictly contain its support and have degree larger than d . The restricted sign function g on the d -cube therefore has nonzero correlation with parity. For some sign $\sigma \in \{-1, 1\}$, the set

$$U = \{x \in \{-1, 1\}^d : g(x) = \sigma x_1 \cdots x_d\}$$

has size greater than 2^{d-1} . Every cube edge with both endpoints in U changes g , since parity changes across each edge.

Define a signed adjacency matrix of the cube recursively by

$$H'_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad H'_{j+1} = \begin{pmatrix} H'_j & I \\ I & -H'_j \end{pmatrix}.$$

Its entrywise absolute value is the ordinary cube adjacency matrix. Induction gives $(H'_j)^2 = jI$, and its trace is zero. Thus H'_d has eigenvalues $\pm\sqrt{d}$, each with multiplicity 2^{d-1} . The $\sqrt{d}$ -eigenspace intersects the space of vectors supported on U nontrivially, because their dimensions sum to more than 2^d . Choose a nonzero real vector v in this intersection, and put $u = |v|$ entrywise. If A_U is the unsigned adjacency induced on U , then

$$u^\top A_U u \geq v^\top H'_d v = \sqrt{d} \|v\|^2 = \sqrt{d} \|u\|^2.$$

All edges of A_U are sensitivity edges of the restriction, and hence of the original function. Extending u by zero to the full input cube gives $\lambda(f) \geq \sqrt{d}$, as required. $\square$

Lemma A.5 ([19]). *Every total Boolean function satisfies $D(f) \leq \text{bs}(f) \deg(f)$.*

Proof. Consider the following deterministic algorithm. If the function restricted by the queries so far is nonconstant, select a monomial of maximum degree in that restricted polynomial and query all its variables. Continue until the restriction is constant.

Fix the actual full input x . In a given round, let S be the selected monomial's support. Fix all other still-unqueried variables to their values in x . The coefficient on S remains nonzero, since no monomial in the current restriction strictly contains S . The resulting function on S is nonconstant. Some assignment to S therefore differs in value from the actual assignment $x|_S$. The nonempty

subset of coordinates on which these two assignments differ is a sensitive block at the original input x .

Different rounds query disjoint sets, so these sensitive blocks are disjoint. There are at most $\text{bs}(f)$ rounds. Restrictions do not increase degree, and each round queries at most $\deg(f)$ variables. The stated deterministic bound follows. $\square$

Proof of Theorem A.1. For nonconstant f , Lemmas A.3 and A.4 give both $\text{bs}(f) = O(Q(f)^2)$ and $\deg(f) = O(Q(f)^2)$. Lemma A.5 then gives $D(f) = O(Q(f)^4)$. Since a deterministic algorithm is also randomized, $R(f) \leq D(f)$. Constant functions require no queries. $\square$

References

- [1] Scott Aaronson and Andris Ambainis. Forrelation: A problem that optimally separates quantum from classical computing. *SIAM Journal on Computing*, 47(3):982–1038, 2018.
- [2] Scott Aaronson, Shalev Ben-David, and Robin Kothari. Separations in query complexity using cheat sheets. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, pages 863–876. ACM, 2016.
- [3] Scott Aaronson, Shalev Ben-David, Robin Kothari, Shravas Rao, and Avishay Tal. Degree vs. approximate degree and quantum implications of Huang’s sensitivity theorem. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 1330–1342. ACM, 2021.
- [4] Andris Ambainis, Kaspars Balodis, Aleksandrs Belovs, Troy Lee, Miklos Santha, and Juris Smotrovs. Separations in query complexity based on pointer functions. *Journal of the ACM*, 64(5):32:1–32:24, 2017.
- [5] Andris Ambainis, Jānis Iraids, and Martins Kokainis. Near-optimal separations of certificate complexity from randomized and quantum query complexity, 2026. arXiv:2609.11664v2.
- [6] Nikhil Bansal and Makrand Sinha. k -Forrelation optimally separates quantum and classical query complexity. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 1303–1316. ACM, 2021.
- [7] Howard Barnum, Michael E. Saks, and Mario Szegedy. Quantum query complexity and semi-definite programming. In *Proceedings of the 18th Annual IEEE Conference on Computational Complexity*, pages 179–193. IEEE, 2003.
- [8] Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca, and Ronald de Wolf. Quantum lower bounds by polynomials. *Journal of the ACM*, 48(4):778–797, 2001.
- [9] Shalev Ben-David and Robin Kothari. Randomized query complexity can beat certificate complexity, 2026. arXiv:2609.15063.
- [10] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997.
- [11] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46(4–5):493–506, 1998.

- [12] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. In Samuel J. Lomonaco, Jr., editor, *Quantum Computation and Quantum Information*, volume 305 of *Contemporary Mathematics*, pages 53–74. American Mathematical Society, 2002.
- [13] Andrew Drucker. Improved direct product theorems for randomized query complexity. *Computational Complexity*, 21(2):197–244, 2012.
- [14] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum, 1996. arXiv:quant-ph/9607014.
- [15] Mika Göös, Toniann Pitassi, and Thomas Watson. Deterministic communication vs. partition number. In *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science*, pages 1077–1088. IEEE, 2015.
- [16] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219. ACM, 1996.
- [17] Hao Huang. Induced subgraphs of hypercubes and a proof of the Sensitivity Conjecture. *Annals of Mathematics*, 190(3):949–955, 2019.
- [18] Nikhil S. Mande, Manaswi Paraashar, and Nitin Saurabh. Randomized and quantum query complexities of finding a king in a tournament. In *43rd IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science*, volume 284 of *Leibniz International Proceedings in Informatics*, pages 30:1–30:19. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023.
- [19] Gatis Midrijanis. Exact quantum query complexity for total Boolean functions, 2004. arXiv:quant-ph/0403168.
- [20] Noam Nisan. CREW PRAMs and decision trees. *SIAM Journal on Computing*, 20(6):999–1007, 1991.
- [21] Noam Nisan and Mario Szegedy. On the degree of Boolean functions as real polynomials. *Computational Complexity*, 4(4):301–313, 1994.
- [22] Ronen Shaltiel. Towards proving strong direct product theorems. *Computational Complexity*, 12(1–2):1–22, 2003.
- [23] Alexander A. Sherstov, Andrey A. Storozhenko, and Pei Wu. An optimal separation of randomized and quantum query complexity. *SIAM Journal on Computing*, 52(2):525–567, 2023. Preliminary version in STOC 2021.
- [24] Robert Špalek and Mario Szegedy. All quantum adversary methods are equivalent. *Theory of Computing*, 2(1):1–18, 2006.
- [25] Xiaoming Sun, Andrew C. Yao, and Shengyu Zhang. Graph properties and circular functions: How low can quantum query complexity go? In *Proceedings of the 19th Annual IEEE Conference on Computational Complexity*, pages 286–293. IEEE, 2004.
- [26] Avishay Tal. Towards optimal separations between quantum and randomized query complexities. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, pages 228–239. IEEE, 2020.

- [27] Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, pages 222–227. IEEE, 1977.